Testing

unittest

Python's **unit testing framework**

Supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

To achieve this, unittest supports some important concepts in an object-oriented way:

Test fixture

A ***test fixture*** represents the preparation needed to perform one or more tests, and any associated cleanup actions. This may involve, for example, creating temporary databases, directories, or starting a server process.

Test case

A **test case** is the individual unit of testing. It checks for a specific response to a particular set of inputs. **unittest** provides a base class, **TestCase**, which may be used to create new test cases.

Test suite

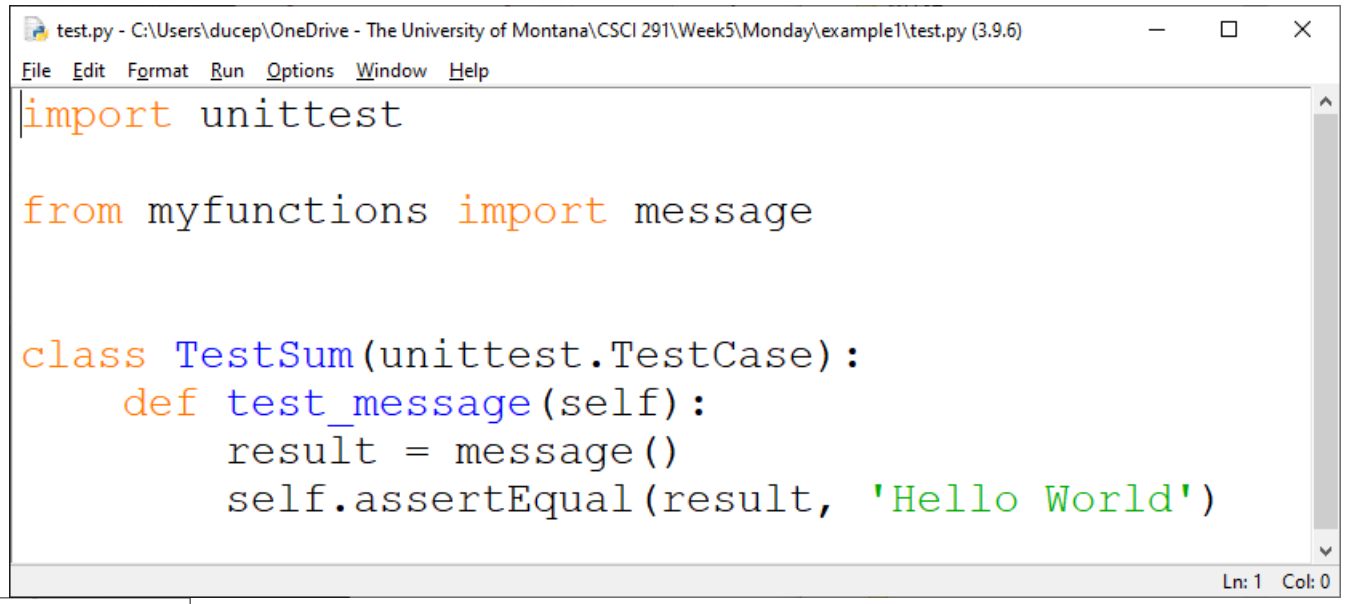
A **test suite** is a collection of test cases, test suites, or both. It is used to aggregate tests that should be executed together.

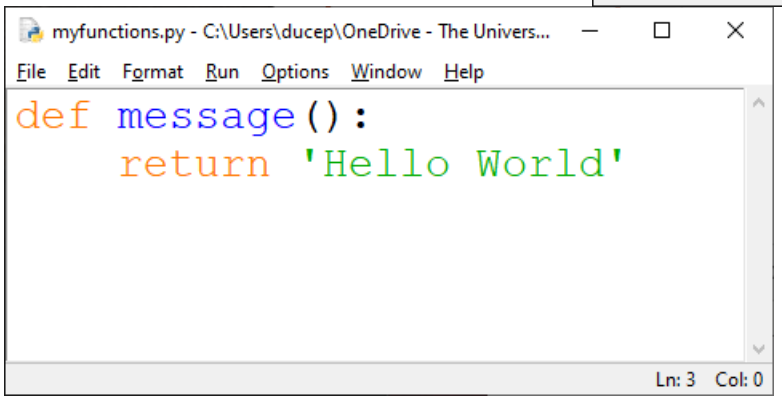
A test suite allows you to categorize test cases in such a way that they match your planning and analysis needs. Do you run functional and performance tests? Create two suites and label them accordingly.

Test runner

A **test runner** is a component which orchestrates the execution of tests and provides the outcome to the user.

Example 1

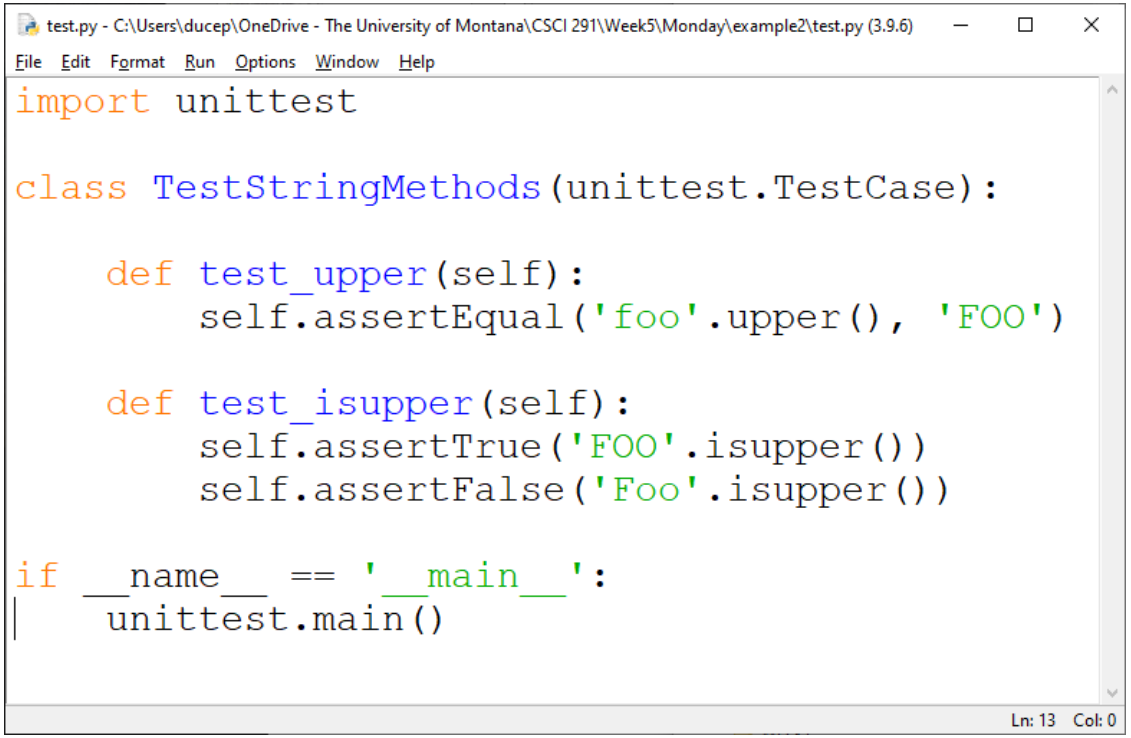

test.py - C:\Users\ducep\OneDrive - The University of Montana\CSCI 291\Week5\Monday\example1\test.py (3.9.6)
File Edit Format Run Options Window Help
import unittest
from myfunctions import message
class TestSum(unittest.TestCase):
def test_message(self):
result = message()
self.assertEqual(result, 'Hello World')
Ln: 1 Col: 0


myfunctions.py - C:\Users\ducep\OneDrive - The Univers...
File Edit Format Run Options Window Help
def message():
return 'Hello World'
Ln: 3 Col: 0

If your file is test.py:

python –m unittest test

Example 2


test.py - C:\Users\ducep\OneDrive - The University of Montana\CSCI 291\Week5\Monday\example2\test.py (3.9.6)
File Edit Format Run Options Window Help
import unittest
class TestStringMethods(unittest.TestCase):
def test_upper(self):
self.assertEqual('foo'.upper(), 'FOO')
def test_isupper(self):
self.assertTrue('FOO'.isupper())
self.assertFalse('Foo'.isupper())
if __name__ == '__main__':
unittest.main()
Ln: 13 Col: 0

```
if __name__ == '__main__’:
  unittest.main()
```

This is a command line entry point. It means that if you execute the script alone by running **python test.py** at the command line, it will call unittest.main(). This executes the test runner by discovering all classes in this file that inherit from unittest.TestCase.

A **testcase** is created by subclassing **unittest.TestCase**. The two individual tests are defined with methods whose names start with the letters test. This naming convention informs the test runner about which methods represent tests.

assertEqual() to check for an expected result

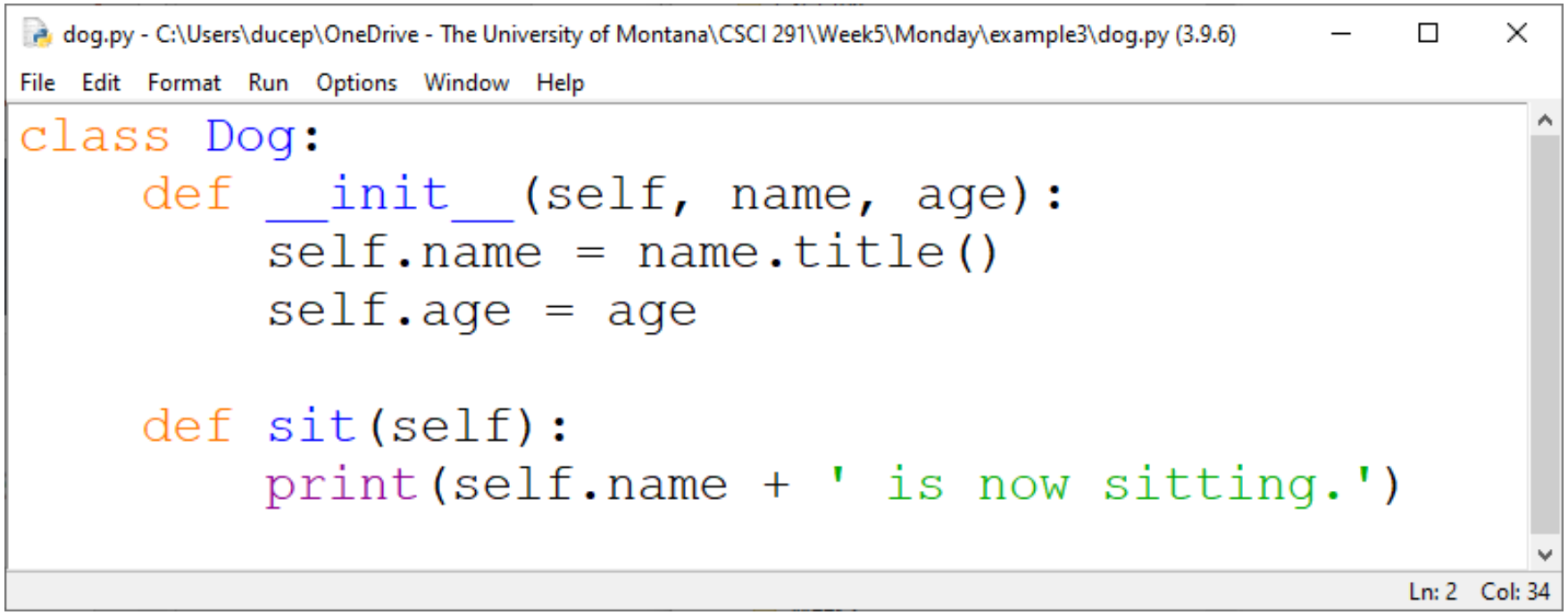
assertTrue() or assertFalse() to verify a condition

These methods are used so the test runner can accumulate all test results and produce a report.

The `TestCase` class provides several assert methods to check for and report failures. The following table lists the most commonly used methods (see the tables below for more assert methods):

Method	Checks that	New in
`assertEqual(a, b)`	`a == b`	
`assertNotEqual(a, b)`	`a != b`	
`assertTrue(x)`	`bool(x) is True`	
`assertFalse(x)`	`bool(x) is False`	
`assertIs(a, b)`	`a is b`	3.1
`assertIsNot(a, b)`	`a is not b`	3.1
`assertIsNone(x)`	`x is None`	3.1
`assertIsNotNone(x)`	`x is not None`	3.1
`assertIn(a, b)`	`a in b`	3.1
`assertNotIn(a, b)`	`a not in b`	3.1
`assertIsInstance(a, b)`	`isinstance(a, b)`	3.2
`assertNotIsInstance(a, b)`	`not isinstance(a, b)`	3.2

Example 3


dog.py - C:\Users\ducep\OneDrive - The University of Montana\CSCI 291\Week5\Monday\example3\dog.py (3.9.6)
File Edit Format Run Options Window Help
class Dog:
def __init__(self, name, age):
self.name = name.title()
self.age = age
def sit(self):
print(self.name + ' is now sitting.')
Ln: 2 Col: 34

The **setUp()** and **tearDown()** methods allow you to define instructions that will be executed before and after each test method.


test.py - C:\Users\ducep\OneDrive - The University of Montana\CSCI 291\Week5\Monday\example3\test.py (3.9.6)
File Edit Format Run Options Window Help
import unittest
import dog
class TestDog(unittest.TestCase):
def setUp(self):
self.dog1 = dog.Dog("Dempsey",3)
def test_name(self):
self.assertEqual(self.dog1.name, "Dempsey")
def test_age(self):
self.assertEqual(self.dog1.age, 3)
if __name__ == '__main__':
unittest.main()
Ln: 1 Col: 0

Django Testing

Django's unit tests uses Python's standard library module: unittest.

Here is an example which subclasses from django.test.TestCase, which is a subclass of unittest.TestCase that runs each test inside a transaction to provide isolation:

```python
from django.test import TestCase
from myapp.models import Animal

class AnimalTestCase(TestCase):
    def setUp(self):
        Animal.objects.create(name="lion", sound="roar")
        Animal.objects.create(name="cat", sound="meow")

    def test_animals_can_speak(self):
        """Animals that can speak are correctly identified"""
        lion = Animal.objects.get(name="lion")
        cat = Animal.objects.get(name="cat")
        self.assertEqual(lion.speak(), 'The lion says "roar"')
        self.assertEqual(cat.speak(), 'The cat says "meow"')
```

There are two methods that you can use for pre-test configuration (for example, to create any models or other objects you will need for the test):

setUpTestData() is called once at the beginning of the test run for class-level setup. You'd use this to create objects that aren't going to be modified or changed in any of the test methods.

setUp() is called before every test method to set up any objects that may be modified by the test (every test method will get a "fresh" version of these objects).

Running tests

Once you've written tests, run them using the test command of your project's manage.py utility:

$ **python manage.py test**

Test discovery is based on the unittest module's built-in test discovery. This will discover tests in any file named "test*.py" under the current project.